Lab.5

1.

Program wypisze trzy razy liczbę 1, czyli: **1, 1, 1**.

Zmienna x jest zmienną lokalną (automatyczną) funkcji f(). Przy każdym wywołaniu funkcji tworzona jest od nowa i inicjalizowana wartością 0. Następnie x++ zwiększa ją do 1 i ta jedynka zostaje wypisana. Po zakończeniu funkcji zmienna jest niszczona, więc kolejne wywołanie znów startuje od 0. Dlatego niezależnie od liczby wywołań wynik to zawsze 1.

2.

Wystarczy dodać słowo kluczowe static przed deklaracją zmiennej: static int x = 0;.

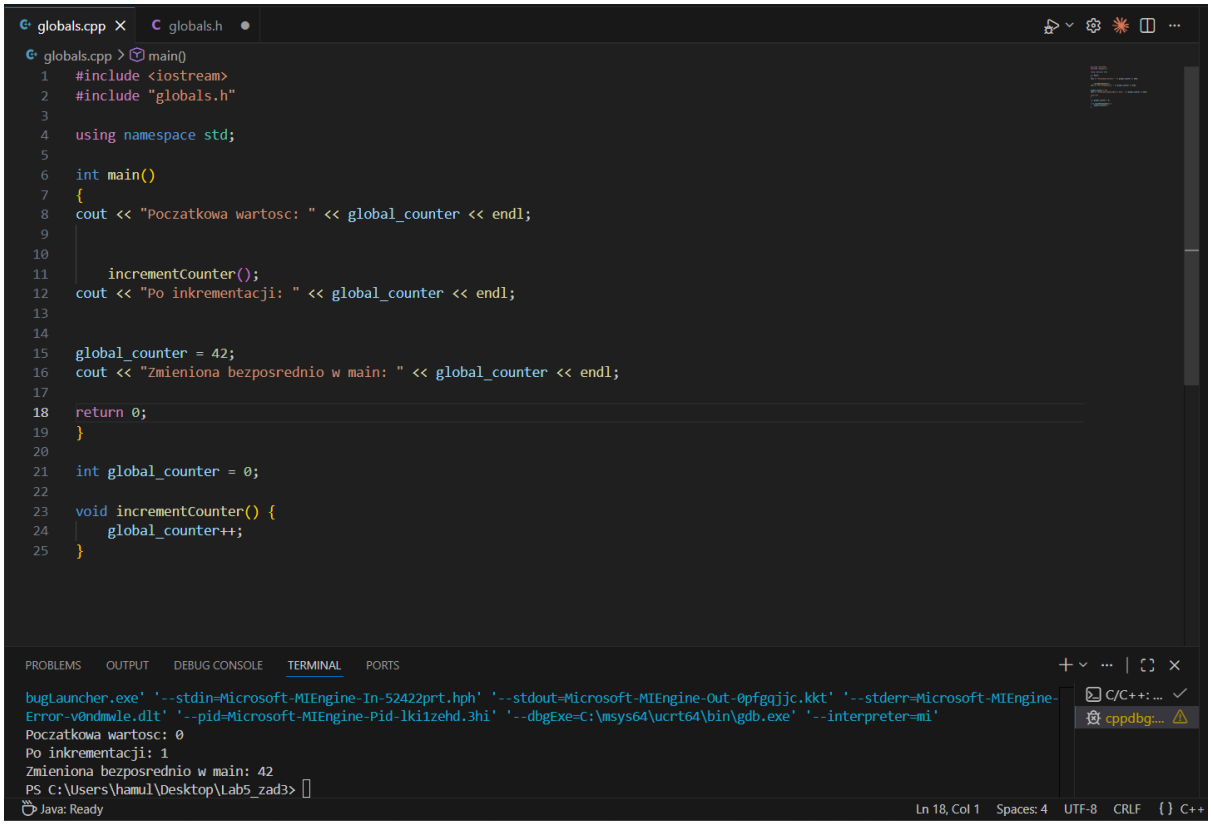
Słowo static nadaje zmiennej statyczny czas życia jest tworzona i inicjalizowana tylko raz (przy pierwszym wywołaniu), a potem istnieje przez cały czas działania programu. Dzięki temu zachowuje swoją wartość między wywołaniami funkcji. Teraz przy kolejnych wywołaniach x rośnie, więc program wypisze **1, 2, 3**.



3.

Słowo kluczowe **extern** informuje kompilator, że dana zmienna jest zdefiniowana w innym pliku . Jest to wyłącznie deklaracja, która nie rezerwuje pamięci, a jedynie mówi: „ta zmienna gdzieś istnieje, połącz ją na etapie linkowania”. Pozwala to bezpiecznie współdzielić jedną zmienną globalną między wieloma plikami.

W naszym przykładzie program składa się z dwoch plików. Plik **globals.h** zawiera deklarację zmiennej (extern int global_counter;), plik **globals.cpp** ją definiuje i alokuje pamięć (int global_counter = 0;), używa jej dołączając plik nagłówkowy.



4.

Program wypisze **10**, a następnie **5**.

W programie istnieją dwie różne zmienne o nazwie x: globalna x = 5 oraz lokalna x = 10 wewnątrz funkcji f(). Wewnątrz f() zmienna lokalna „przesłania” (shadowing) zmienną globalną, dlatego cout wypisuje 10. W funkcji main nie ma lokalnego x, więc używana jest zmienna globalna i wypisane zostaje 5.

5.

W programie występują trzy zmienne, każda o innej charakterystyce:

Zmienna	Zasięg	Czas życia
global	globalny cały plik	cały czas działania programu (statyczny)
local	lokalny tylko wewnątrz f()	automatyczny tworzona i niszczona przy każdym wywołaniu
s	lokalny tylko wewnątrz f()	statyczny istnieje przez cały program, zachowuje wartość

W efekcie global jest zawsze 10, local zawsze 5 (tworzone od nowa), a s rośnie między wywołaniami. Dlatego wynik to **10 5 1**, a potem **10 5 2**.

6.

Nie, ten kod nie ma sensu jest błędny. Funkcja f() tworzy lokalną zmienną x (automatyczną), a następnie zwraca jej adres (&x).

Problem polega na tym, że zmienna lokalna jest niszczona w chwili zakończenia funkcji. Zwrócony wskaźnik wskazuje więc na obszar pamięci, który już do niej nie należy to tzw. **wiszący wskaźnik (dangling pointer)**. Odczyt spod tego adresu to niezdefiniowane zachowanie. Kompilator g++ od razu ostrzega: „warning: address of local variable 'x' returned”.

Jak naprawić: zwrócić wartość przez kopię (typ int zamiast int*), przekazać wskaźnik na zmienną z zewnątrz, albo użyć zmiennej static lub pamięci dynamicznej (new).

7.

Funkcja increment(int *p) przyjmuje wskaźnik, a operacja (*p)++ zwiększa o 1 wartość znajdującą się pod tym adresem czyli oryginalną zmienną, a nie jej kopię.

Przetestowałem funkcję na trzech klasach pamięci: zmiennej **automatycznej** (lokalna), zmiennej **statycznej** (static) oraz **globalnej**. We wszystkich przypadkach funkcja działa tak samo operuje na rzeczywistym adresie zmiennej, więc każda zostaje zwiększona o 1 (11, 51, 101). Klasa pamięci nie wpływa na działanie funkcji, ponieważ i tak przekazujemy adres zmiennej, a nie jej wartość.

